

Тема: Понятие класса и объекта

В ООП очень **важно предварительное проектирование**. В общей сложности можно выделить следующие этапы разработки объектно-ориентированной программы:

Формулирование задачи.

Определение объектов, участвующих в ее решении.

Проектирование классов, на основе которых будут создаваться объекты. В случае необходимости установление между классами наследственных связей.

Определение ключевых для данной задачи свойств и методов объектов.

Создание классов, определение их полей и методов.

Создание объектов.

Решение задачи путем организации взаимодействия объектов.

Спроектировать и реализовать программно-аппаратную модель системы для заданной предметной области (например, «Умный лифт», «Автомат самообслуживания» или «Система климат-контроля»).

Цель работы — перейти от написания линейных скриптов к созданию **многослойной архитектуры**, отвечающей современным промышленным стандартам разработки на Java.

Техническое задание

1. Архитектурное разделение. Программа должна быть строго разделена на три части:

- **Domain (Сущность):** Класс, содержащий только логику и состояние. Он **запрещает** использование `Scanner` и `System.out.println`. Вся информация передается через возвращаемые значения.
- **DTO/Result (Обмен):**
 - Методы, выполняющие действия, должны возвращать **объект-результат** (например, `record OperationResult`), содержащий:
 1. `boolean isSuccess` (удалось ли действие).
 2. `String message` (пояснение для пользователя).
 - UI обязан проанализировать этот результат и вывести пользователю соответствующее уведомление (Успех/Отказ).
- **UI/Application (Интерфейс):** Единственное место, где разрешен ввод/вывод. Этот слой «дёргает» методы Домена и показывает результат пользователю.

2. Требования к Доменной сущности (Класс-модель)

- **Инкапсуляция:** Все поля — `private`. Доступ к изменению состояния — только через публичные методы с валидацией.
- **Валидация (Self-Defense):** Объект должен сам проверять корректность входных данных (границы, логические запреты) и не позволять переводить себя в «невозможное» состояние.
- Использование **Enum** для состояний, имеющих фиксированный набор значений (статус двери, режим работы, состояние двигателя).

3. Обработка ошибок (LBYL vs Exceptions)

- **Валидация ввода:** Интерфейс должен использовать стратегию *Look Before You Leap* (проверка `hasNextInt()`/`hasNextDouble()`). Ввод буквы вместо цифры не должен приводить к падению программы.
- **Бизнес-отказы:** Если действие невозможно по правилам задачи (нет денег, закрыто, перегруз) — возвращаем `isSuccess = false`.
- **Критические сбои:** Использование `throw new IllegalStateException()` только там, где нарушение логики говорит о «поломке» самой программы (инварианты безопасности).

4. Чистота кода (Clean Code)

- **Guard Clauses:** В методах логики сначала идут проверки на ошибки с ранним `return`, и только в конце — выполнение полезного действия (избегайте «лестниц» из `if-else`).
- **Single Responsibility:** Один метод делает ровно одну вещь. Метод `move()` не должен одновременно спрашивать вес у пользователя.

5. Структура Меню (Слой UI). Интерфейс должен обеспечивать полный цикл управления объектом. Обязательные пункты:

- **Цикл управления:** Наличие бесконечного цикла с пунктом выхода.
- **Обработка «Пустого объекта»:** Проверка на `null` перед вызовом методов сущности.
- **Инициализация (Пункт «Создать/Настроить»):** Первым делом пользователь должен создать экземпляр сущности с заданными параметрами. До создания объекта все функциональные пункты меню должны выдавать предупреждение: «Сначала создайте объект».
- **Мониторинг (Пункт «Статус»):** Вывод текущего состояния объекта (всех его ключевых полей) через переопределенный метод `toString()`.
- **Функциональные действия:** Пункты меню, которые напрямую вызывают методы бизнес-логики задачи (например: «Выполнить операцию», «Изменить режим», «Добавить ресурс»).
- **Завершение (Пункт «Выход»):** Чистый выход из цикла программы.

Резюме (как это должно выглядеть в ТЗ):

- **Реализовать сущность** с защищенными полями и логикой валидации.
- **Реализовать UI**, обеспечивающий ввод/вывод и защиту от ошибок пользователя.
- **Связать их через Result-объекты**, исключив прямую печать текста внутри логики.
- **Доказать автономность**, продемонстрировав работу всех пунктов меню.

Контрольный вопрос для самопроверки:

«Если я захочу заменить консоль на другое приложение (Telegram-бот или веб-сайт), сколько строк кода в классе моей **сущности** мне придется изменить?»

Правильный ответ: «Ни одной».

Индивидуальные задания

1. Система «Умный Кондиционер»

- **Поля:**
 1. `int currentTemp` — текущая температура в комнате (в градусах).
 2. `int targetTemp` — желаемая температура, которую должен поддерживать прибор.
 3. `boolean isEcoMode` — режим экономии энергии (вкл/выкл).
- **Методы:**
 1. `setTemperature(int newTemp)` — устанавливает `targetTemp`. Если `newTemp` ниже 16 или выше 30 градусов, вывести ошибку и не менять значение.
 2. `powerBoost()` — если режим `isEcoMode` включен, выключить его и установить `targetTemp` на 18 градусов для быстрого охлаждения.

2. Стиральная машина

- **Поля:**
 1. `double maxWeight` — максимально допустимый вес белья (в кг).
 2. `double currentWeight` — вес белья, уже загруженного в барабан.
 3. `boolean isWaterLeaking` — датчик протечки (`true`, если обнаружена вода на полу).
- **Методы:**
 1. `addLaundry(double weight)` — добавляет вес к `currentWeight`. Если сумма превышает `maxWeight`, вывести: «Перегрузка барабана!».
 2. `startCycle()` — запускает стирку. Если `isWaterLeaking` равно `true`, отменить запуск и вывести: «Ошибка: обнаружена протечка!».

3. Автоматическая Кофемашина

- **Поля:**
 1. `int waterLevel` — остаток воды в баке (в мл).
 2. `int beansLevel` — остаток кофейных зерен (в граммах).
 3. `int wasteContainer` — заполненность контейнера для отходов (в порциях, макс. — 10).
- **Методы:**
 1. `makeCoffee(int waterNeeded, int beansNeeded)` — если воды и зерен достаточно, а `wasteContainer` меньше 10 — уменьшить запасы, увеличить счетчик отходов на 1 и вывести: «Кофе готов!».
 2. `cleanContainer()` — обнуляет `wasteContainer` и выводит: «Контейнер очищен».

4. Робот-пылесос

- **Поля:**
 1. `int batteryCharge` — уровень заряда батареи (в %).
 2. `boolean isDustBinFull` — статус пылесборника (заполнен или нет).
 3. `String currentRoom` — название комнаты, где находится робот.
- **Методы:**
 1. `moveToRoom(String roomName)` — меняет `currentRoom`. Метод работает только если `batteryCharge` больше 15%. В противном случае вывести: «Низкий заряд, еду на базу».

2. `startCleaning()` — если `isDustBinFull` истинно, вывести: «Очистите контейнер!». Иначе — начать уборку и снизить заряд на 10%.

5. Цифровой Спидометр

- **Поля:**
 1. `int currentSpeed` — текущая скорость (км/ч).
 2. `int speedLimit` — установленное ограничение скорости для данной дороги.
 3. `boolean cruiseControl` — статус круиз-контроля (вкл/выкл).
- **Методы:**
 1. `accelerate(int amount)` — увеличивает скорость. Если `currentSpeed` становится выше `speedLimit`, вывести: «Внимание: превышение!».
 2. `toggleCruise()` — включает круиз-контроль, но только если `currentSpeed` больше 40 км/ч.

6. Электронная Библиотека

- **Поля:**
 1. `int totalBooks` — общее количество книг в библиотеке.
 2. `int booksOut` — количество книг, выданных на руки.
 3. `boolean isLibraryOpen` — статус (открыта/закрыта).
- **Методы:**
 1. `issueBook()` — если библиотека открыта и есть свободные книги (`totalBooks > booksOut`), увеличить `booksOut` на 1.
 2. `returnBook()` — уменьшить `booksOut`, если оно больше 0. Если библиотека закрыта, вывести: «Оставьте книгу в корзине у входа».

7. Спортивный Трекер

- **Поля:**
 1. `int stepsCount` — количество пройденных шагов за день.
 2. `int dailyGoal` — цель по шагам (например, 10000).
 3. `double caloriesBurned` — сожженные калории.
- **Методы:**
 1. `addSteps(int steps)` — добавляет шаги к `stepsCount`. Если цель `dailyGoal` достигнута, вывести: «Цель выполнена!».
 2. `resetDay()` — обнуляет шаги и калории для нового дня.

8. Микроволновая печь

- **Поля:**
 1. `int secondsLeft` — время до конца нагрева (в секундах).
 2. `int powerLevel` — мощность нагрева (от 100 до 800).
 3. `boolean doorClosed` — состояние дверцы.
- **Методы:**
 1. `setTimer(int sec)` — устанавливает время. Если `sec` меньше 0, вывести: «Неверное время».
 2. `pressStart()` — если `doorClosed` истинно и `secondsLeft > 0`, вывести: «Нагрев пошел». Иначе вывести причину ошибки.

9. Торговый автомат (Вендинг)

- **Поля:**
 1. `int itemCount` — количество товара (например, бутылок воды).
 2. `double depositedMoney` — сумма денег, внесенная пользователем.
 3. `double itemPrice` — цена за одну единицу товара.
- **Методы:**
 1. `insertMoney(double amount)` — увеличивает `depositedMoney`.
 2. `buyItem()` — если товара больше 0 и денег достаточно, уменьшить `itemCount`, вычесть цену из `depositedMoney` и выдать сдачу (обнулить баланс).

10. Почтовый ящик

- **Поля:**
 1. `int lettersCount` — текущее количество писем в ящике.
 2. `int boxCapacity` — максимальная вместимость ящика.
 3. `boolean isLocked` — закрыт ли ящик на ключ.
- **Методы:**
 1. `receiveLetter()` — если `lettersCount < boxCapacity`, прибавить 1 письмо. Иначе вывести: «Ящик переполнен, письмо торчит наружу».
 2. `emptyBox(boolean hasKey)` — если `hasKey` совпадает со статусом `isLocked`, обнулить `lettersCount`. Если ключа нет — вывести: «Доступ запрещен».

11. Система «Умная теплица»

- **Поля:**
 1. `double humidity` — текущая влажность воздуха (в %).
 2. `double waterLevel` — запас воды в системе полива (в литрах).
 3. `boolean ventsOpen` — состояние форточек для проветривания (`true` — открыты).
- **Методы:**
 1. `irrigate(double volume)` — если `waterLevel` достаточно, уменьшить запас воды и увеличить `humidity`. Если воды мало — вывести: «Ошибка: низкий уровень воды».
 2. `ventilate(double outsideTemp)` — если `outsideTemp > 25`, установить `ventsOpen = true`. Если ниже — закрыть, чтобы не выстудить растения.

12. Контроллер «Электросамокат»

- **Поля:**
 1. `int batteryLevel` — заряд батареи (0–100%).
 2. `int currentSpeed` — текущая скорость (км/ч).
 3. `boolean headlightOn` — состояние фары.
- **Методы:**
 1. `accelerate(int power)` — если `batteryLevel > 5`, увеличить скорость и снизить заряд. Если заряд критический — ограничить скорость до 5 км/ч.
 2. `toggleLight()` — включает фару, но если заряд меньше 10%, фара не включается для экономии хода.

13. Модуль «Умная кормушка для питомца»

- **Поля:**
 1. `double foodWeight` — вес корма в контейнере (в граммах).
 2. `int portionsToday` — количество выданных порций за текущие сутки.
 3. `int portionLimit` — суточный лимит порций (например, 4).
- **Методы:**
 1. `feed(double amount)` — если `portionsToday < portionLimit` и в контейнере хватает корма, выдать порцию и увеличить счетчик. Иначе — вывести «Лимит исчерпан».
 2. `refill(double weight)` — пополняет контейнер. Если `weight <= 0`, вывести ошибку некорректного значения.

14. Терминал «Платная парковка»

- **Поля:**
 1. `int freeSpaces` — количество свободных мест на стоянке.
 2. `double hourlyRate` — стоимость одного часа парковки.
 3. `boolean isOpen` — статус въезда (открыт/закрыт).
- **Методы:**
 1. `enterCar()` — если `freeSpaces > 0` и `isOpen`, уменьшить количество мест на 1. Если мест нет — автоматически сменить `isOpen` на `false`.
 2. `updateRate(double newRate)` — меняет стоимость часа, но только если `newRate > 0`, иначе оставляет старую цену.

15. Система «Пожарная сигнализация»

- **Поля:**
 1. `int smokeLevel` — уровень задымленности (условные единицы).
 2. `boolean alarmActive` — включена ли сирена.
 3. `int sprinklersStatus` — состояние распылителей воды (0 — выкл, 1 — ожидание, 2 — полив).
- **Методы:**
 1. `checkSensor(int sensorValue)` — обновляет `smokeLevel`. Если значение выше 50, установить `alarmActive = true` и `sprinklersStatus = 2`.
 2. `resetSystem()` — если `smokeLevel` пришел в норму (меньше 10), выключить сирену и вернуть распылители в режим ожидания.

16. Контроллер «Промышленный холодильник»

- **Поля:**
 1. `double internalTemp` — температура внутри камеры.
 2. `boolean compressorRunning` — работает ли компрессор.
 3. `int doorOpenTime` — время в секундах, в течение которого дверь открыта.
- **Методы:**
 1. `monitorDoor(int seconds)` — увеличивает `doorOpenTime`. Если время больше 60 секунд, вывести: «Внимание: потеря холода!».
 2. `coolDown()` — если `internalTemp` выше нормы, запустить компрессор (`true`). Если температура достигла целевой — выключить для экономии.

17. Приложение «Аренда велосипедов»

- **Поля:**
 1. `double userBalance` — баланс кошелька пользователя.
 2. `boolean isBikeLocked` — заблокирован ли замок велосипеда.
 3. `int rideMinutes` — длительность текущей поездки.
- **Методы:**
 1. `unlockBike(double deposit)` — если `userBalance >= deposit`, сменить `isBikeLocked` на `false` и списать депозит. Иначе — «Недостаточно средств».
 2. `endRide(double pricePerMinute)` — заблокировать замок и рассчитать итоговую стоимость, вычтя её из остатка депозита.

18. Модуль «Автоматические ворота»

- **Поля:**
 1. `String state` — текущее состояние ("Closed", "Opening", "Open", "Closing").
 2. `boolean obstacleDetected` — датчик препятствия (`true`, если в проеме что-то есть).
 3. `int remoteSignal` — код сигнала от пульта (должен совпадать с кодом системы).
- **Методы:**
 1. `pressButton(int signalCode)` — если код верный и `obstacleDetected` ложно, начать открытие ворот. Если код неверный — ничего не делать.
 2. `safetyStop()` — если во время закрытия (`Closing`) датчик обнаружил объект, немедленно остановить ворота и сменить состояние на "Opening".

19. Инфо-табло «Очередь в банке»

- **Поля:**
 1. `int lastTicketNumber` — номер последнего выданного талона.
 2. `int currentServingTicket` — номер талона, который сейчас обслуживается.
 3. `int activeWindows` — количество работающих окон операторов.
- **Методы:**
 1. `issueTicket()` — увеличивает `lastTicketNumber` на 1 и выводит: «Ваш номер в очереди: [X]».
 2. `callNext()` — если `currentServingTicket < lastTicketNumber`, увеличить номер обслуживаемого талона. Если все обслужены — вывести «Очередь пуста».

20. Система «Авиабилет»

- **Поля:**
 1. `double ticketPrice` — базовая цена билета.
 2. `int baggageWeight` — вес багажа (в кг).
 3. `boolean isBusinessClass` — тип места (`true` — бизнес).
- **Методы:**
 1. `calculateTotal()` — возвращает итоговую цену. Если `baggageWeight > 20`, прибавить штраф. Если бизнес-класс — умножить цену на 2.
 2. `updateBaggage(int newWeight)` — меняет вес багажа, но выдает предупреждение, если вес превышает 32 кг (запрет на перевозку).

21. Контроллер «Умное окно»

- **Поля:**

1. `int openingPercentage` — процент открытия окна (от 0 до 100).
2. `boolean rainSensor` — датчик дождя (`true` — идет дождь).
3. `boolean heatingOn` — статус отопления в комнате.

- **Методы:**

1. `setPosition(int percent)` — устанавливает степень открытия. Если `rainSensor` истинно, не позволяет открывать окно более чем на 5% и выводит: «Дождь! Окно заблокировано».
2. `autoClose()` — если `heatingOn` становится `true`, автоматически устанавливает `openingPercentage` в 0 для сохранения тепла.

22. Модуль «Электронный ошейник»

- **Поля:**

1. `double distanceToBase` — расстояние собаки от базы (в метрах).
2. `double safeRadius` — радиус безопасной зоны.
3. `boolean signalActive` — статус предупреждающего звукового сигнала.

- **Методы:**

1. `updateLocation(double newDistance)` — обновляет `distanceToBase`. Если новое расстояние больше `safeRadius`, включает `signalActive` и выводит: «Внимание: выход из зоны!».
2. `setSafeZone(double newRadius)` — меняет размер зоны, но только если `newRadius` больше 5 метров (техническое ограничение).

23. Счётчик «Умный турникет»

- **Поля:**

1. `int peopleInside` — количество людей в здании.
2. `int maxCapacity` — максимальная вместимость здания.
3. `boolean emergencyBlock` — экстренная блокировка всех входов.

- **Методы:**

1. `grantEntry()` — если `emergencyBlock` ложно и `peopleInside` < `maxCapacity`, прибавить 1 человека. Иначе вывести: «Вход запрещен».
2. `processExit()` — уменьшает `peopleInside` на 1, если в здании кто-то есть. Если людей стало 0, выводит: «Здание пусто».

24. Система «Автоматическая теплица»

- **Поля:**

1. `double soilMoisture` — влажность почвы (в %).
2. `double tankVolume` — остаток воды в баке для полива.
3. `boolean pumpError` — ошибка насоса (`true`, если сломан).

- **Методы:**

1. `startWatering(double waterNeeded)` — если влажность меньше 30% и нет ошибки насоса, уменьшить воду в баке и увеличить влажность.
2. `checkPump()` — имитирует диагностику. Если `tankVolume` упал до 0, устанавливает `pumpError = true` (работа «всухую»).

25. Бортовой компьютер «Электробус»

- **Поля:**

1. `int batteryCharge` — уровень заряда (%).
2. `int passengersCount` — количество пассажиров.
3. `double energyConsumption` — текущий расход энергии.

- **Методы:**

1. `addPassengers(int count)` — добавляет людей. При этом `energyConsumption` должно увеличиваться пропорционально весу (например, +0.1 за каждого).
2. `drive(int km)` — уменьшает заряд батареи на `km * energyConsumption`. Если заряда не хватает на путь, вывести: «Маршрут невозможен, требуется зарядка».

26. Контроллер «Промышленный пресс»

- **Поля:**

1. `double pressure` — текущее давление (в барах).
2. `double maxPressure` — критический порог давления.
3. `boolean safetyShield` — защитный экран (`true` — опущен).

- **Методы:**

1. `applyPressure(double step)` — увеличивает давление. Если `safetyShield` ложно, не начинать работу и вывести: «Опустите экран!».
2. `emergencyRelease()` — мгновенно сбрасывает `pressure` до 0 и открывает экран.

27. Игровой автомат «Слот-машина»

- **Поля:**

1. `int credits` — количество кредитов у игрока.
2. `int jackpot` — текущая сумма джекпота.
3. `boolean isMachineBroken` — статус поломки.

- **Методы:**

1. `spin(int bet)` — если `credits >= bet` и автомат исправен, вычесть ставку и прибавить её часть к джекпоту. Вывести: «Барабаны крутятся».
2. `winJackpot()` — если выпала удача, прибавить весь `jackpot` к `credits` и обнулить джекпот.

28. Система «Автоматическая парковка»

- **Поля:**

1. `int occupiedSpaces` — занятые места.
2. `int totalSpaces` — всего мест на парковке.
3. `double dailyRevenue` — выручка за день.

- **Методы:**

1. `parkCar(double fee)` — если есть места, увеличить `occupiedSpaces` и прибавить `fee` к выручке.
2. `releaseSpace()` — уменьшить количество занятых мест, если они больше 0.

29. Контроллер «Подогрев сидений»

- **Поля:**

1. `int heatLevel` — уровень нагрева (0, 1, 2, 3).
2. `boolean passengerPresent` — датчик наличия пассажира.
3. `int batteryVoltage` — напряжение в сети авто.

- **Методы:**

1. `increaseHeat()` — увеличивает уровень нагрева. Если `passengerPresent` ложно, оставить уровень 0 и вывести: «Сиденье пустое».
2. `checkBattery()` — если `batteryVoltage` падает ниже 11, принудительно установить `heatLevel = 0` для сохранения энергии на запуск мотора.

30. Модуль «Сейф с таймером»

- **Поля:**

1. `boolean isLocked` — закрыт ли замок.
2. `int hourToOpen` — час, в который сейф можно открыть (например, 9 утра).
3. `int currentHour` — текущее время системы.

- **Методы:**

1. `tryOpen()` — если `currentHour` совпадает или больше `hourToOpen`, сменить `isLocked` на `false`. Иначе вывести: «Доступ закрыт до [X] часов».
2. `changeOpenTime(int newHour)` — меняет время открытия, но только если сейф уже открыт (`isLocked == false`).